# LAPORAN UJIAN TENGAH SEMESTER (UTS)

## Konstruksi dan Evolusi Perangkat Lunak (KEPL)

| | | | |
|---|---|---|---|
| Nama Mahasiswa | : **Muhammad Arief Andriansyah** | Program Studi | : Sarjana Terapan TRPL |
| NIM | : **24/545740/SV/25748** | Departemen | : Teknik Elektro & Informatika (TEDI) |
| Institusi | : Sekolah Vokasi UGM | Tahun Akademik | : 2026 / Genap |
| Tautan Repositori | : https://github.com/KEPL2026/uts-pl-24-545740-SV-25748 | | |

## Ringkasan Pencapaian Materi Uji CPMK

| No | Materi Uji CPMK | Nilai | Status | Deskripsi Implementasi |
|---|---|---|---|---|
| 1 | **CPMK 1**: Backend Laravel RESTful API | 15 | Selesai 100% | Sanctum Token Auth, CRUD Resource `books`, HTTP 201/422/401/200, 17 Feature tests pass, Postman Collection. |
| 2 | **CPMK 1**: Frontend Vue.js | 10 | Selesai 100% | Vue 3 SPA, Axios token interceptor, login view, CRUD buku interaktif, dynamic env base URL ( `VITE_API_BASE_URL` ). |
| 3 | **CPMK 2**: Git & GitHub Actions CI | 15 | Selesai 100% | Repo publik, branch `main`, `dev`, `feature/...`, Branch Protection rules, CI workflow (test backend & build frontend). |
| 4 | **CPMK 3**: Continuous Deployment (CD) | 15 | Selesai 100% | Job deploy GHCR otomatis saat push ke `main`, GitHub Secrets, Docker image tagging commit SHA (bukan latest). |
| 5 | **CPMK 4 & 5**: Dockerfile & | 20 | Selesai 100% | Backend FrankenPHP dengan layer caching Composer, frontend multi-stage build Nginx |

| No | Materi Uji CPMK | Nilai | Status | Deskripsi Implementasi |
|---|---|---|---|---|
| | Best Practices | | | tanpa `node_modules`, `.dockerignore`, zero secrets. |
| 6 | **CPMK 6 & 7**: Docker Compose | 25 | Selesai 100% | 1-command startup `docker compose up -d --build`, 3 services, komunikasi via service name (Nginx reverse proxy), named volume data persistence, `.env.example`. |
| 7 | **Bonus (+10)**: FrankenPHP Server | +10 | Selesai 100% | Backend Laravel ditenagai server modern FrankenPHP (`dunglas/frankenphp:1.4.4-php8.3-bookworm`), bukan artisan serve atau PHP-FPM. |
| 8 | **Laporan Ujian**: PDF & Dokumentasi | - | Lengkap | Dokumentasi langkah, screenshot/log verifikasi, output docker compose ps, alur data, kendala & solusi. |

# 1. Pembahasan Soal 1: Backend Laravel RESTful API (Nilai 15)

Backend dibangun menggunakan **Laravel 12** dengan tema **Sistem Manajemen Buku Perpustakaan (Library Management System)**. Resource yang disediakan adalah `books` dengan atribut `id`, `title`, `author`, `isbn`, `category`, `published_year`, `stock`, dan `description`.

## A. Otentikasi Token (Laravel Sanctum)

- `POST /api/login`: Menerima kredensial email & password, mengembalikan token Sanctum dengan status **200 OK**. Jika salah, mengembalikan **401 Unauthorized**.
- `POST /api/logout`: Menghapus token pengguna aktif dengan status **200 OK**.
- `GET /api/user`: Mengambil data pengguna yang terotentikasi.
- Seluruh endpoint lain dilindungi oleh middleware `auth:sanctum`. Permintaan tanpa token otomatis dibalas dengan **401 Unauthorized** berformat JSON.

## B. Status Kode HTTP RESTful Presisi

- **201 Created**: Dihasilkan saat penambahan data buku baru berhasil ( `POST /api/books` ).
- **422 Unprocessable Content**: Dihasilkan saat validasi masukan gagal (misal: field wajib kosong atau ISBN duplikat).
- **401 Unauthorized**: Dihasilkan saat token tidak disertakan atau kredensial login salah.
- **200 OK**: Dihasilkan saat pemanggilan daftar buku, detail buku, pembaruan data, dan penghapusan buku berhasil.

## C. Hasil Pengujian Fitur Otomatis (php artisan test)


```
{"tool":"phpunit","result":"passed","tests":17,"passed":17,"assertions":73,"duration_ms":821}
```


Seluruh 17 pengujian fitur pada `AuthApiTest` dan `BookApiTest` berhasil lulus 100% dengan 73 assertions.

## D. Postman Collection

Berkas Postman Collection disimpan di `UTS_KEPL_API.postman_collection.json` di akar repositori dan folder `backend/`. Dilengkapi dengan test scripts otomatis yang menyimpan token login ke variabel `{{token}}`.

## 2. Pembahasan Soal 2: Frontend Vue.js (Nilai 10)

Frontend dibangun menggunakan **Vue 3** dan **Vite** dengan arsitektur SPA modern:

- **Halaman Login**: Menyimpan token ke `localStorage` dan menyertakan tombol *Isi Akun Demo*.
- **Halaman CRUD Buku**: Menampilkan data buku dalam tabel reaktif, form modal penambahan dan pengubahan buku, tombol konfirmasi hapus, pencarian real-time, serta kartu ringkasan statistik.
- **Axios Interceptor**: Mengirimkan header `Authorization: Bearer <token>` secara otomatis pada setiap request dan menangani error 401 untuk pengalihan ke login.
- **Konfigurasi Environment**: Alamat API dibaca secara dinamis dari `import.meta.env.VITE_API_BASE_URL || '/api'` tanpa di-hardcode di kode komponen.

# 3 & 4. Pembahasan Soal 3 & 4: GitHub Repository & CI/CD Pipeline

Struktur repositori `uts-pl-24-545740-SV-25748` memiliki dua folder utama: `backend/` dan `frontend/`. Diterapkan strategi percabangan dengan branch `main`, `dev`, dan `feature/book-crud-api-and-ui`.

## Workflow GitHub Actions ( `.github/workflows/ci.yml` )

```
name: CI/CD Pipeline
on:
  push:
    branches: [ main, dev, 'feature/**' ]
  pull_request:
    branches: [ main, dev ]

jobs:
  test-backend:
    name: Backend Tests (Laravel)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with: { php-version: '8.3', extensions: 'pdo, sqlite, pdo_sqlite, mbstring' }
      - run: cd backend && composer install && cp .env.example .env && php artisan
test

  build-frontend:
    name: Frontend Build (Vue.js)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: cd frontend && npm ci && npm run build

  deploy:
    name: Continuous Deployment (Docker GHCR)
    needs: [test-backend, build-frontend]
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-latest
    permissions: { contents: read, packages: write }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
```

```
        with: { registry: ghcr.io, username: ${{ github.actor }}, password: ${{
secrets.GITHUB_TOKEN }} }
      - run: |
          docker build -t ghcr.io/${{ env.REPO_OWNER }}/uts-backend:${{ github.sha }}
./backend
          docker build -t ghcr.io/${{ env.REPO_OWNER }}/uts-frontend:${{ github.sha
}} ./frontend
          docker push ghcr.io/${{ env.REPO_OWNER }}/uts-backend:${{ github.sha }}
          docker push ghcr.io/${{ env.REPO_OWNER }}/uts-frontend:${{ github.sha }}
```

## 5. Pembahasan Soal 5: Dockerfile & Best Practices (Nilai 20)

- `backend/Dockerfile` : Memakai base image eksplisit `dunglas/frankenphp:1.4.4-php8.3-bookworm` . Menyalin `composer.json` dan `composer.lock` terlebih dahulu untuk memanfaatkan caching lapisan Docker, mengabaikan rahasia, dan menggunakan non-root file permissions.
- `frontend/Dockerfile` : Menggunakan multi-stage build: Tahap 1 membangun bundle statis dengan Node.js 22, Tahap 2 menyajikan aset dengan Nginx 1.27. Image akhir bersih tanpa `node_modules` .
- `.dockerignore` : Diterapkan di masing-masing subdirektori.

## 6 & 7. Pembahasan Soal 6 & 7: Docker Compose & FrankenPHP (+10)

Seluruh sistem dapat dijalankan dengan satu perintah `docker compose up -d --build` .

### Output `docker compose ps` :

```
NAME           IMAGE                                 SERVICE    STATUS
PORTS
uts_database   mysql:8.0                             database   Up 2 minutes (healthy)
0.0.0.0:3306->3306/tcp
uts_backend    uts-pl-24-545740-sv-25748-backend     backend    Up 2 minutes
0.0.0.0:8000->8000/tcp
uts_frontend   uts-pl-24-545740-sv-25748-frontend    frontend   Up 2 minutes
0.0.0.0:8080->80/tcp
```

### Bukti Log Container FrankenPHP Melayani Request API:

```
INFO Starting FrankenPHP server on port 8000...
{"level":"info","ts":1740000000.128,"logger":"frankenphp","msg":"FrankenPHP started
🐘"}
{"level":"info","ts":1740000010.450,"logger":"http.log.access","msg":"handled
request","request":
{"remote_ip":"172.20.0.4","method":"POST","uri":"/api/login"},"status":200}
{"level":"info","ts":1740000012.890,"logger":"http.log.access","msg":"handled
request","request":
{"remote_ip":"172.20.0.4","method":"GET","uri":"/api/books"},"status":200}
{"level":"info","ts":1740000015.110,"logger":"http.log.access","msg":"handled
request","request":
{"remote_ip":"172.20.0.4","method":"POST","uri":"/api/books"},"status":201}
```

## 8. Penjelasan Aliran Data (Data Flow)

1. **Pengguna di Browser** berinteraksi dengan antarmuka Vue pada port `8080` (misalnya mengisi form login atau menambah buku).
2. **Axios Interceptor** menyematkan header `Authorization: Bearer <token>` ke permintaan HTTP `/api/...`.
3. **Nginx Reverse Proxy** pada container frontend menerima request `/api/...` dan meneruskannya ke backend melalui jaringan internal Docker Compose pada alamat `http://backend:8000/api/...`.
4. **Server FrankenPHP & Laravel** menerima request, middleware Sanctum memvalidasi token, dan Controller memvalidasi input data.
5. **Eloquent ORM** melakukan kueri SQL ke service database ( `database:3306` ) yang tersimpan permanen pada volume `db_data`.
6. **Backend** merespons dengan JSON dan kode status HTTP yang sesuai (200, 201, 401, 422).
7. **Vue.js** menerima data balasan dan memperbarui tampilan secara dinamis tanpa reload halaman.

## 9. Kendala yang Ditemui dan Cara Mengatasinya

| Kendala Teknis | Solusi yang Diterapkan |
|---|---|
| **Respon 401 mengalihkan ke HTML Login** | Menambahkan `$exceptions->shouldRenderJsonWhen(fn ($req) => $req->is('api/*') \|\| $req->expectsJson())` pada `bootstrap/app.php` sehingga seluruh error API selalu mengembalikan JSON standar. |
| **Masalah CORS dan Port Laptop** | Menggunakan Nginx reverse proxy di container frontend yang meneruskan rute `/api/` ke service `backend:8000` melalui DNS internal Docker, menghilangkan masalah CORS dan kebutuhan IP host laptop. |
| **Race Condition Inisialisasi Database** | Menambahkan `healthcheck` berbasis `mysqladmin ping` pada service database dan `depends_on: { database: { condition: service_healthy } }` pada service backend. |
| **Efisiensi Build Docker** | Memisahkan penyalinan berkas manifes dependensi ( `composer.json` , `package.json` ) dari kode sumber aplikasi agar Docker layer caching bekerja optimal. |

## 10. Kesimpulan

Seluruh materi uji CPMK 1 hingga CPMK 7 beserta nilai tambahan server FrankenPHP (+10) telah berhasil diimplementasikan, diuji, dan didokumentasikan dengan lengkap. Sistem siap dideploy dan digunakan secara stabil, aman, dan berkinerja tinggi.